



CODEXUS
TECHNOLOGIES

PROJECT BRIEF

StockSense

AI-Powered Smart Inventory Monitoring & Management System

"Every bin knows what it holds." | Designed & Made in Sri Lanka

Prepared for	Kavindu Hasaranga – Electronics and IoT Trainee
Date of issue	03 October 2026
Internship period	6 months (01 October 2026 – 31 March 2027)

Welcome to Codexus Technologies

Dear Kavindu,

Congratulations, and welcome to the Codexus Technologies team! We are delighted to have you join us as an **Electronics and IoT Trainee**. Over the next six months you will not be doing “practice exercises”, you will be designing a real product that we intend to manufacture, sell in Sri Lanka and export under the proud label **“Made in Sri Lanka”**.

This document explains your internship terms, the main project you will own (**StockSense**), why we chose it, and a clear, step by step plan.

This project will also be the work you showcase to your university and the highlight of your career portfolio. Ask questions often, document everything, and aim for excellence.

Warm regards,

Anjana Suraweera

Codexus Technologies

1 Project Overview – StockSense

StockSense is a low-cost smart-shelf and smart-bin system. Each storage bin sits on a small sensor platform that **weighs its contents continuously, identifies the item with RFID/QR**, and reports the exact quantity in real time to a cloud dashboard. An analytics engine then learns consumption patterns, warns about low stock, flags suspicious discrepancies (loss, theft, mis-picks), predicts the date an item will run out, and recommends **when and how much to reorder**.

1.1 The problem we are solving

Pain point	What happens today in most Sri Lankan businesses
Manual stock counting	Staff count items by hand, which is slow, expensive and full of human error.
Stock-outs & over-stocking	Money is lost either because a key item runs out or because cash is locked in excess stock.
Hidden losses	Theft, wastage and mis-picks are only discovered weeks later during a stock-take.
Expired / damaged goods	Pharmacies, food outlets and labs lose stock to expiry or poor storage conditions (heat, humidity).
Unaffordable imports	Imported smart-inventory systems are priced for large corporations and are not designed for local conditions.

1.2 Our solution

Sense	Load cells measure weight; RFID/QR tags identify the item; temperature and humidity sensors monitor storage conditions.
Think (at the edge)	A custom ESP32/STM32 PCB filters the signals, calculates quantity (weight ÷ unit weight), detects events locally and keeps working even when the internet is down.
Connect	Data is sent securely to a cloud/server using MQTT over Wi-Fi, with local buffering and automatic re-send.
See	A web dashboard shows stock levels, movements, alerts, environment and history – in English, Sinhala and Tamil.
Predict	An analytics engine detects low stock and discrepancies, predicts run-out dates and recommends reorder timing and quantity.

Why this is a strong “Made in Sri Lanka” export product

- Hardware is very cheap to build, while the value of the solution (hardware + dashboard + AI insights) is high.
- Designed for local realities: power cuts, unstable internet, humid climate, multi-language staff and small-business budgets.
- Recurring revenue is possible: sell the node once and charge a monthly dashboard/analytics subscription.
- The same product serves many industries and many countries with only a software configuration change.

2 Market Opportunity & Business Value

StockSense is a single platform that can be sold to many types of customers. Phase 1 customers are local; export is the long-term goal.

Target customer	What StockSense does for them	Example items tracked
Pharmacies & clinics	Tracks stock, expiry-sensitive items and storage temperature; alerts before stock-out.	Medicine boxes, consumables
Garment & manufacturing factories	Monitors small parts and accessories at production lines; stops line stoppages.	Buttons, zips, labels, screws
Hotels, restaurants & cafés	Kitchen and bar stock monitoring; wastage and pilferage detection.	Rice, spices, beverages
Supermarkets & retail shops	Real-time shelf and back-store levels; automated reorder suggestions.	Packaged goods
Tea, spice & food exporters	Warehouse bin tracking with humidity monitoring for quality control.	Bulk spice/tea containers
Labs, workshops & hardware shops	Never run out of critical consumables or spare parts.	Chemicals, fasteners, parts

Export potential: South Asia, the Maldives, the Middle East and Africa all have large numbers of small and medium businesses that cannot afford premium imported systems but still need accurate stock control.

2.1 Cost vs. value (indicative – to be validated in Month 3)

Item	Indicative figure	Comment
Manufacturing cost per node (prototype)	About LKR 8,000 – 12,000	Falls significantly at volume production
Target selling price per node	About LKR 28,000 – 45,000	Depends on capacity (5 kg / 20 kg / 50 kg)
Dashboard & analytics subscription	Monthly fee per site	Recurring revenue – the main profit engine
Customer saving	Reduced stock-outs, losses & manual labour	Easy ROI story for every sale

Your Month-3 costing work (full Bill of Materials, supplier prices, packaging and shipping) will replace these estimates with real numbers.

3 System Architecture

StockSense has five layers. You will design and build every one of them.

1. SENSE	2. EDGE NODE	3. CONNECT	4. CLOUD	5. INSIGHT
Load cells + HX711 ADC RFID (SPI) / QR tag Temp & humidity (I ² C)	Custom ESP32 / STM32 PCB Filtering & calibration Local event detection Offline buffer	Wi-Fi + MQTT (TLS) UART / I ² C / SPI on- board OTA updates	MQTT broker REST API Time-series database Authentication	Web dashboard Alerts (email / SMS / WhatsApp) Forecasting & reorder AI

Data flows left to right: bin → edge node → network → cloud → decisions.

3.1 Key product features

Feature	Description
Real-time quantity	Weight-based counting per bin, accurate after calibration (target $\pm 1\%$).
Item identification	RFID tag or QR code on each bin links it to an item, unit weight and reorder rules.
Movement log	Every addition or removal is recorded with time, amount and (optionally) who took it.
Smart alerts	Low stock, unusual removal, possible theft, sensor fault, over-temperature or high humidity.
Offline-first	Node keeps measuring and stores data during internet or power interruptions, then syncs automatically.
Battery backup	Keeps the node alive and sending an “outage” event during power cuts.
Consumption analytics	Learns daily/weekly patterns, predicts the run-out date and suggests reorder quantity.
Multi-language dashboard	English, Sinhala and Tamil – ideal for local staff and for export markets.
Over-the-air updates	Firmware can be improved remotely after the product is sold.

4 Technical Scope

Area	What you will use / build
Hardware	- Custom PCB (designed in KiCad) based on ESP32 (main) – schematic, layout, Gerber files, assembly and bring-up - Load cell (5 / 20 / 50 kg) with HX711 or similar 24-bit ADC; RFID reader (e.g., MFRC522/PN532); temperature & humidity sensor; status LEDs, buzzer, small display - Power: 5 V input, protection circuits, Li-ion battery backup with charger/protection, low-power design 3D-printed enclosure and load-cell mounting platform
Firmware	- Languages & tools: C/C++ with PlatformIO / Arduino framework, ESP-IDF where needed - Arduino for quick sensor experiments, ESP32 as production target, STM32 port of the core firmware as a low-power / future variant - Calibration, filtering (median + moving average / Kalman), event detection, offline queue, OTA, watchdog and error handling
Communication	- Wi-Fi (with easy provisioning), MQTT with TLS and authentication, topic design and QoS UART (debug / optional GSM or barcode module), I²C (sensors, display), SPI (RFID, flash/SD) - Optional stretch: BLE provisioning or LoRa for large warehouses
Cloud & backend	- MQTT broker (Mosquitto / EMQX), REST API (FastAPI or Node.js), PostgreSQL / TimescaleDB (or InfluxDB) - User accounts, multi-site / multi-bin data model, secure device registration - Containerised with Docker, deployable on a low-cost VPS
Web dashboard	- UI/UX designed in Figma, built with React (or similar) + charts - Pages: Overview, Bin detail, Movement log, Alerts, Environment, Reports, Settings
Analytics / AI	- Python: low-stock rules, anomaly detection (discrepancies), demand forecasting (moving average, exponential smoothing / Holt-Winters, simple ML) - Reorder recommendation: reorder point and quantity from lead time, consumption rate and safety stock
Quality & documentation	Test plans, calibration reports, bug log, version control with Git, user manual, datasheet

5 Roadmap – 6 Months at a Glance

Period	Phase	Main outcome
Month 1 Oct 2026	Phase 1A – Research & Requirements	Market/competitor study, requirements, architecture, tool setup
Month 2 Nov 2026	Phase 1B – Design & Simulation	Complete design package: schematic, PCB draft, firmware design, backend, dashboard UI, simulated end-to-end demo
Month 3 Dec 2026	Phase 2 – Hardware Prototype	Working breadboard prototype: weighing, RFID, sensors, MQTT live
Month 4 Jan 2027	Phase 3 – Custom PCB	PCB fabricated, assembled, tested; firmware running on it; STM32 port started
Month 5 Feb 2027	Phase 4 – Integration & Intelligence	Live dashboard, real data, forecasting & reorder engine, enclosure
Month 6 Mar 2027	Phase 5 – Pilot, Documentation & Showcase	Pilot test, reliability tests, university report, final demo, product datasheet

Gate reviews

Gate 1 – end of Month 2: design package review with Anjana.

Gate 2 – end of Month 4: custom PCB working and publishing live data over MQTT.

Gate 3 – mid Month 6: pilot results and final demo readiness.

6 Phase 1 (Months 1–2): Research, Design & Planning

For the first two months you need **no hardware**, only your laptop. The goal is to arrive at Month 3 with every design decision made, every part chosen and a working **simulated** version of the whole system.

6.1 Month 1 – Research, requirements & architecture

Week	Focus	Tasks	Deliverables
Week 1	Onboarding & problem research	- Set up laptop: Git/GitHub, KiCad, VS Code + PlatformIO, Docker, Python, Node.js, Figma - Study how load cells, strain gauges and 24-bit ADCs work; read RFID, MQTT basics - Research the inventory problem in pharmacies, factories and shops	Tool-setup checklist Learning notes
Week 2	Market & competitor study	- Study existing smart-shelf / smart-inventory products (features, price, weaknesses) - Create 3–4 customer personas and the top 10 use-cases - Prepare a short online survey / call script for 5 potential local customers	Competitor comparison table Personas & use-cases
Week 3	Requirements	- Write the requirements document (functional, performance, cost, environment, power) - Define measurable targets (accuracy, latency, battery backup, cost per node) - Build a risk register (technical, supply, schedule)	Requirements spec (SRS) KPI target sheet
Week 4	System architecture	- Draw block diagrams: hardware, firmware, data flow, cloud and security - Select MCU, sensors, ADC, power design and cloud stack with a written justification - Design the MQTT topic structure and data/JSON message format	Architecture document Component shortlist MQTT & data spec

6.2 Month 2 – Detailed design & simulation

Week	Focus	Tasks	Deliverables
Week 5	Hardware design	- Draw the full schematic in KiCad (MCU, ADC, RFID, sensors, power, battery backup, protection) - Simulate analog/power parts (LTspice or similar) and verify the load-cell front-end and power budget - Create the first costed Bill of Materials with supplier links	KiCad schematic Simulation results Costed BOM v1
Week 6	PCB & enclosure design	- Complete PCB layout (placement, routing, ground, antenna keep-out, DFM checks) - View in 3D; prepare Gerber files (not yet ordered) - Design the enclosure and load-cell platform in FreeCAD/Fusion	PCB layout v1 3D enclosure model

Week	Focus	Tasks	Deliverables
Week 7	Firmware & cloud design	- Firmware architecture: state machine, calibration, filtering, offline queue, OTA - Simulate ESP32 firmware in Wokwi publishing fake sensor data over MQTT - Build the cloud stack on your laptop with Docker: broker, database, REST API	Firmware design doc Simulated node code Running local backend
Week 8	Dashboard, AI prototype & Gate 1	- Design dashboard screens in Figma, then build a working version with simulated data - Generate realistic consumption data and prototype forecasting & anomaly detection in Python - Connect everything in simulation: simulated node → MQTT → database → dashboard → forecast - Compile the Design Package and present to Anjana (Gate 1)	Dashboard v1 (simulated) Forecasting prototype Design Package Gate 1 presentation

6.3 Free tools you will use in Phase 1

Electronics	KiCad (schematic & PCB), LTspice (analog/power simulation), FreeCAD (enclosure)
Firmware	VS Code + PlatformIO, Wokwi online ESP32/Arduino simulator, Git/GitHub
Cloud & data	Docker, Mosquitto/EMQX, PostgreSQL/TimescaleDB, FastAPI or Node.js, MQTT Explorer
Dashboard & AI	Figma, React (or similar), Grafana (optional), Python (pandas, scikit-learn, statsmodels)
Documentation	Markdown/Word, draw.io for diagrams, weekly report template

Phase 1 exit criteria (Gate 1)

- Complete KiCad schematic and PCB layout reviewed
- Final BOM with real prices and chosen suppliers
- Firmware design and a simulated node working with MQTT
- Cloud backend and dashboard demo running on your laptop with simulated data
- Forecasting and low-stock/discrepancy detection demonstrated on simulated data
- Written Design Package approved by Codexus

7 Phases 2–5 (Months 3–6): Build, Integrate, Prove

Month	Objective	Key activities	Deliverables
Month 3 Hardware prototype	Make it work on a breadboard	- Purchase approved parts; set up lab bench - Weighing: wire load cell + ADC with Arduino, then ESP32; calibrate and measure accuracy - Add RFID, temperature/humidity, display, buzzer - Connect to Wi-Fi and publish real data over MQTT to your backend	Working prototype Calibration & accuracy report
Month 4 Custom PCB	Move to your own board	- Order PCB; assemble (soldering), inspect and bring up board by board - Port firmware to the PCB; test power, battery backup, I²C/SPI/UART buses - Troubleshoot and revise to PCB v2 if needed - Begin STM32 port of the core firmware	Assembled PCB Bring-up log STM32 proof-of-concept
Month 5 Integration & intelligence	Connect everything for real	- Live dashboard with real node data; alerts via email/SMS/WhatsApp - Train and tune forecasting, anomaly detection and reorder recommendation with real consumption data - Security hardening (TLS, device authentication), OTA update test - Print enclosure and build final-looking unit(s)	Complete working system Enclosure & platform Test report
Month 6 Pilot & showcase	Prove it in real use	- Pilot at a real or realistic site (e.g., a mock store/pharmacy shelf) for 2+ weeks - Reliability tests: power cut, Wi-Fi loss, humidity, long run - Final documentation: user manual, datasheet, source code, university report - Final demo to Codexus and preparation for university presentation	Pilot results Final report & demo video Product datasheet Full source & design files

8 Success Targets & Evaluation

8.1 Technical targets (to be refined in Month 1)

Target	Goal
Weight/quantity accuracy after calibration	Within $\pm 1\%$ of the true value
Event-to-dashboard latency	Alert visible within about 5 seconds of the event
Data delivery	99%+ of readings delivered, none lost during short internet outages (buffered)

Target	Goal
Power cut behaviour	Node stays online on battery for at least 8 hours and reports the outage
Forecast quality	Run-out predictions with average error under about 15% on test data
Manufacturing cost	Meets the target cost per node set in the BOM

8.2 How your work will be evaluated

Area	Weight	What we look for
Hardware & PCB design	20%	Correct, clean, well-documented circuits; successful assembly and troubleshooting
Firmware quality	20%	Reliable, readable, tested code on ESP32 (and STM32 port)
Communication & cloud	15%	Stable MQTT, secure, well-designed API and database
Dashboard & analytics	20%	Clear usable dashboard; working low-stock, discrepancy, forecast and reorder features
Integration, testing & documentation	15%	End-to-end working system with test evidence and complete documents
Professionalism & initiative	10%	Punctuality, weekly reporting, communication, learning attitude, teamwork

8.3 Final deliverables

Working StockSense prototype	At least one complete smart bin + dashboard + analytics, demonstrated live
Design package	Schematics, PCB files, BOM, enclosure, firmware and software source code
Final internship/project report	Suitable for university submission (after Codexus review)
Showcase material	Presentation slides, poster, and a short demo video
Product documents	User manual and a one-page product datasheet

9 Working Rhythm & Ground Rules

Daily	Short 10-minute update with Anjana (what you did, what you will do, blockers).
Weekly	Written progress report every Friday: completed tasks, plans, issues, learning.
Monthly	Formal review of milestones, deliverables and feedback.
Documentation	Keep a daily engineering logbook and commit all work to the company Git repository.

A final word

This is a chance to build something the country can be proud of. If you plan well in the first two months, build carefully in the next four and document everything, StockSense will be a strong university project, a great portfolio piece and a real product with a real market.

We are excited to see what you create, Kavindu. Welcome aboard!